

Mathematical Foundations of Computer Graphics & Vision



Mingyang Song

Computer Graphics Laboratory, ETH Zürich
DisneyResearch|Studios

Today!

- Exercises admin
- Handout, exercise 1

Admin

- 6 graded exercises (50% of the grade in total)
- Oral examination (50% of the grade)
- Individual work
- You will have 2 weeks minimum each time
- Programming in Python
- Written report (or a single Jupyter notebook)

Admin

- Exercise sessions will be
 - short presentation of the new handout
 - Q&A session
 - possibly some lecture theory recap (will get announced before)

Admin

- Read the handout carefully
- Write a good report
 - Quality over quantity
 - Jupyter notebook or a PDF
 - Derivations in LaTeX

Admin

- Use the Moodle forum to ask for help
- Participate & help others
- *Do not post solution code there*
(but describing your implementation is fine)
- Administrative questions → send me an email
- *Copying code (either from other students or from external sources) is strictly prohibited!*

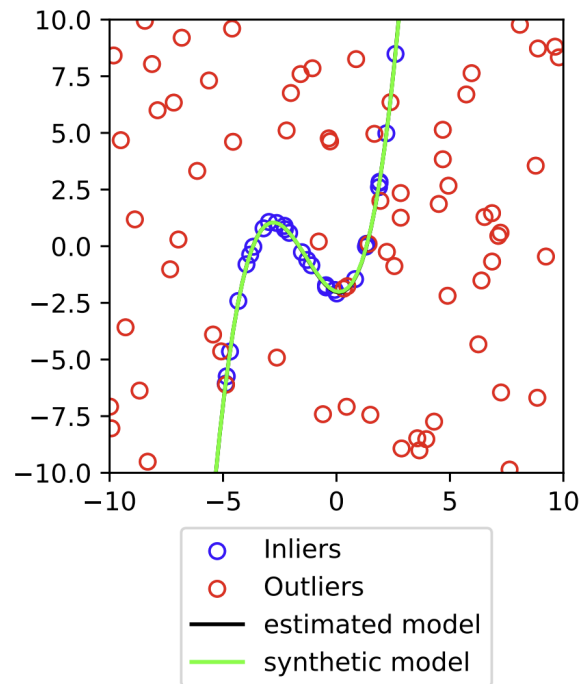
Submissions

- On Moodle
- Deadlines are on Tuesdays 23:59, strict late policy:
 - **Less than 1h:** 100% (no penalty)
 - **1h - 12h:** 80% (00:59:01 -> penalty applies)
 - **12h - 24h:** 60%
 - **more than 24h:** 0% (not graded)

Questions?

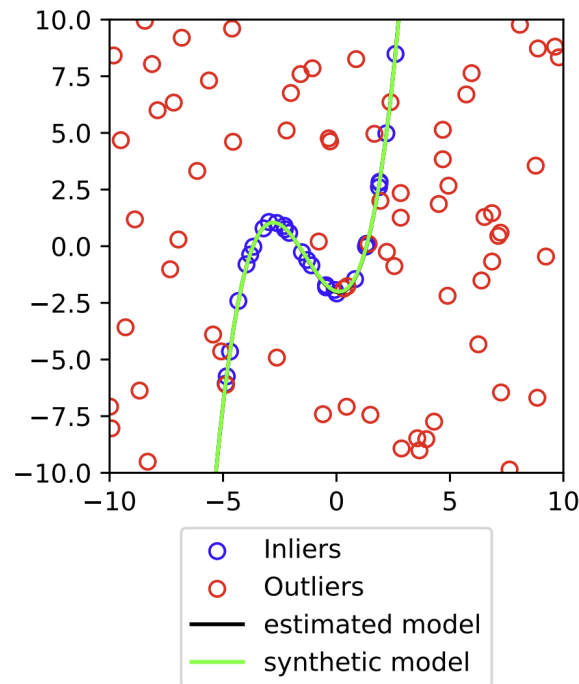
Exercise 1.1

- Implement RANSAC as seen in the lecture last week
- Use your implementation to fit a polynomial on randomly generated points (synthetic data)



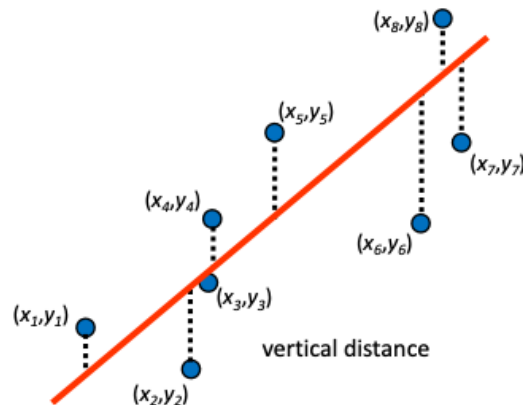
Exercise 1.1

- First step, implement synthetic data generation -> Ground truth solution exists!
- Polynomial of the form $y = \sum_{i=0}^n \alpha_i x^i$
- Generate $N = 100$ points in the domain $[-10, 10] \times [-10, 10]$
- Some points are inliers, they are on the line, but have noise
- Some points are outliers, they are not on the line at all



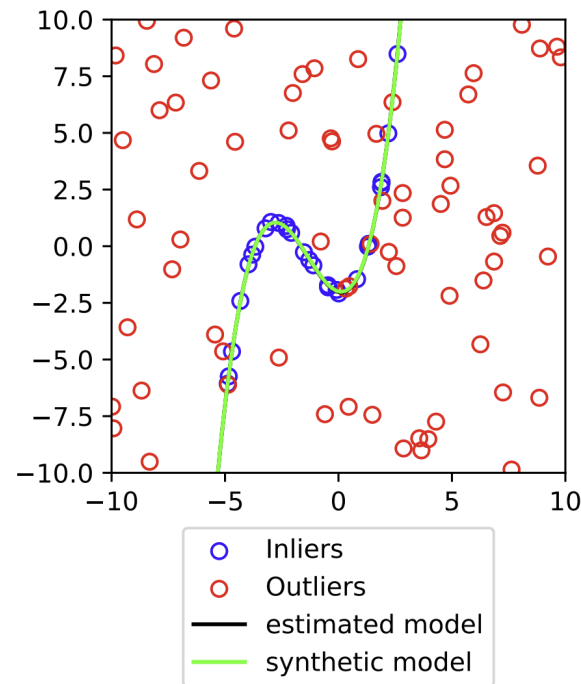
Exercise 1.1

- Inlier if within τ from the line
- Vertical (algebraic) cost
 - $p = (p_x, p_y)$
 - $d(f, p) = p_y - f(p_x)$



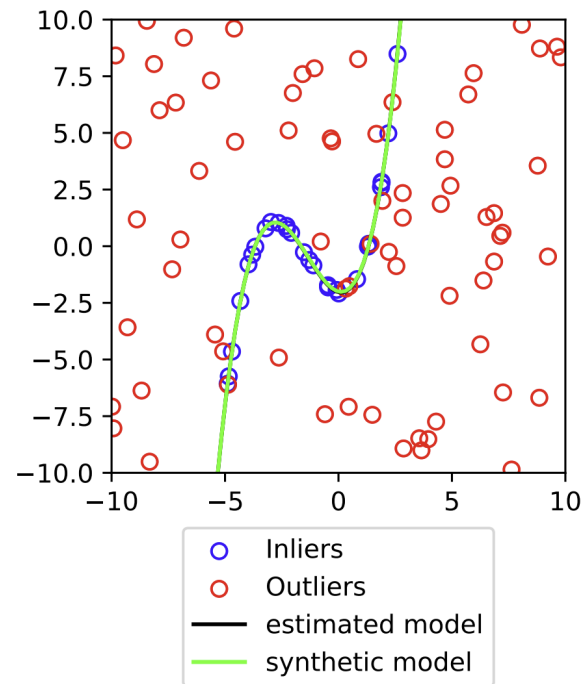
Exercise 1.1

- Make sure that the synthesized inliers are indeed inliers and the synthesized outliers are indeed outliers (and within the domain)
- Total number of points is always N ($N = 100$). So if the ratio of outliers is $r = 10\%$, then 10 points are outliers and 90 points are inliers. If $r = 30\%$, then 30 outliers and 70 inliers.



Exercise 1.1

- Implement RANSAC as seen in the lecture
- Inputs are
 - The points (Nx2) numpy array
 - Number of iterations
 - The inlier threshold
 - Sample size
- Outputs are
 - The coefficients of the estimated model
 - The inliers/outliers



Exercise 1.1

- Automatically compute the number of RANSAC iterations

$$N = \frac{\log(1-p)}{\log(1-(1-\epsilon)^s)}$$

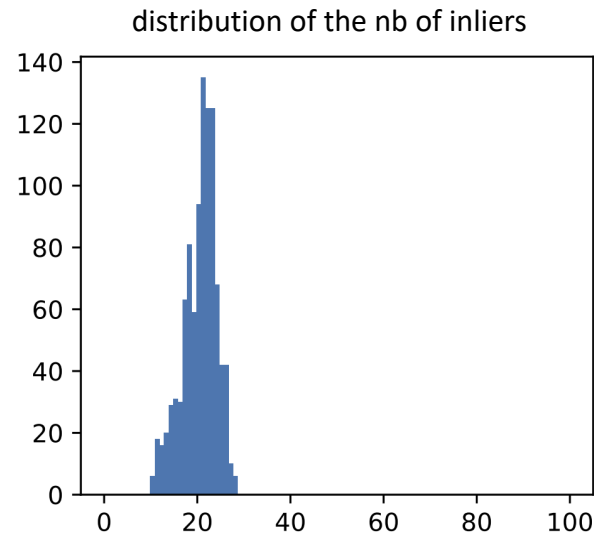
probability that at least one sample has no outliers (usually set to 0.99)

Outlier ratio

*Sample size (minimum data)
e.g. $s=2$ for line fitting, $s=3$ for circle*

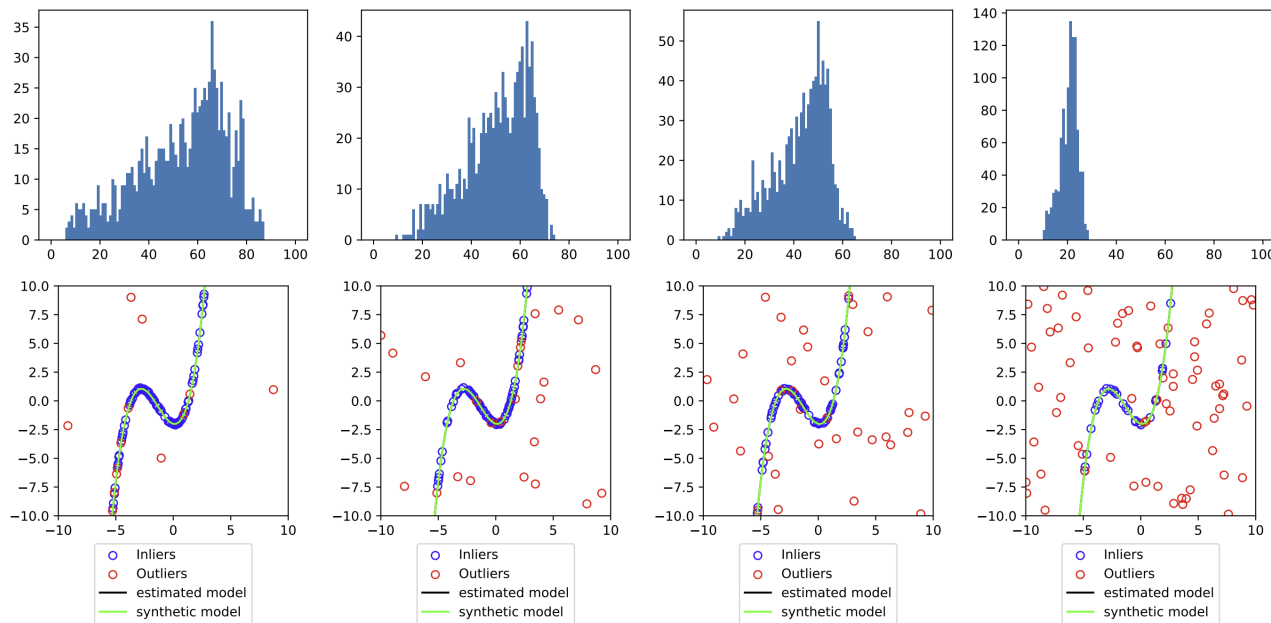
Exercise 1.1

- Plot results for data sets with $r = 5, 20, 30, 70\%$ of outliers
- Run RANSAC several times (1000) and plot the distribution of the best number of inliers for each run on a histogram
- Compare with exhaustive search



Exercise 1.1

- A successful task looks like similar to this:



Numpy Vectorization

- Python is slow^[1], use *numpy* vectorization to make it better

```
def is_close_to_target(points, target, threshold):  
    is_close = np.zeros((points.shape[0],), dtype=bool)  
  
    for i in range(points.shape[0]):  
        diff = points[i] - target  
        is_close[i] = np.linalg.norm(diff) <= threshold  
  
    return is_close
```

N = 10M, runtime = 43.5s

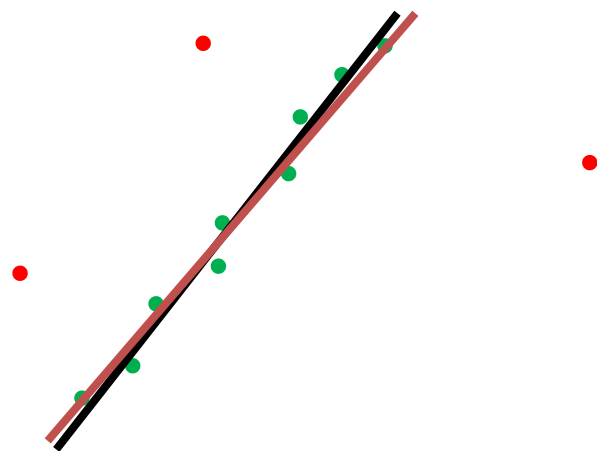
```
def is_close_to_target(points, target, threshold):  
    is_close = np.zeros((points.shape[0],), dtype=bool)  
  
    diff = points - target[None]  
    is_close = np.linalg.norm(diff, axis=1) < threshold  
  
    return is_close
```

N = 10M, runtime = 0.7s

[1] <http://jakevdp.github.io/blog/2014/05/09/why-python-is-slow/>

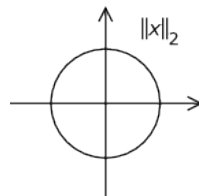
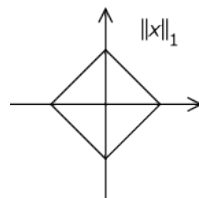
Exercise 1.2

- Line fitting
- IRLS (L1)
- Linear Programming (L1, L-inf)

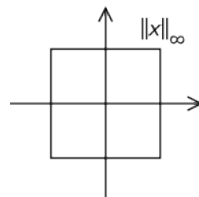


Exercise 1.2

$$|x|_1 = \sum_{i=1}^n |x_i|$$



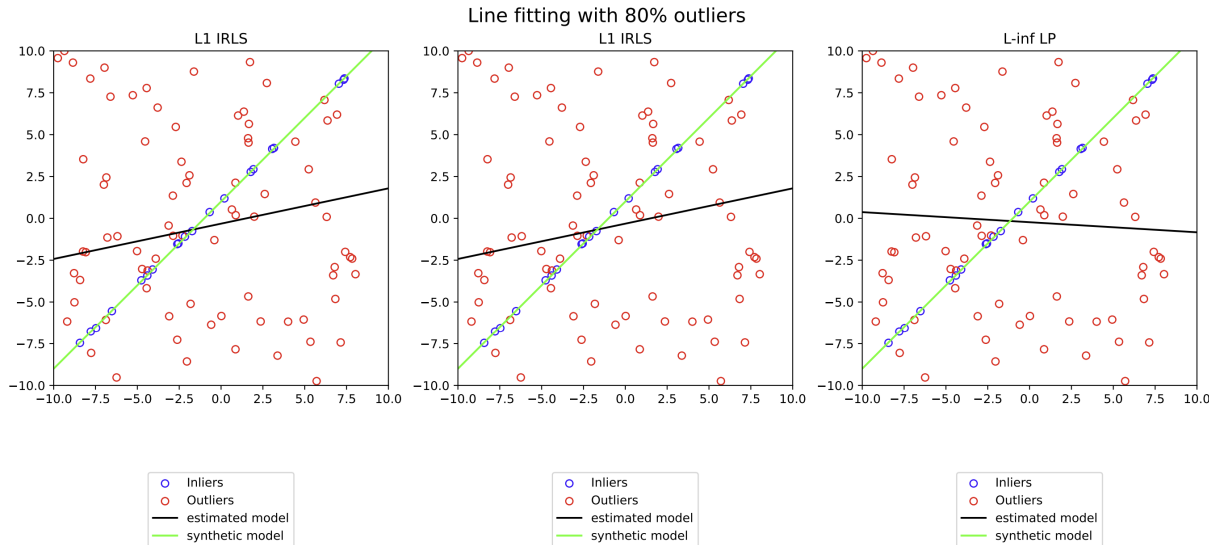
$$|x|_\infty = \max_i |x_i|$$



Source - wikipedia

Exercise 1.2

- A successful result looks like this (80% outliers)



Jupyter notebook

- Providing a Jupyter notebook template



The screenshot shows a Jupyter notebook titled "jupyter Exercise1 (unsaved changes)". The notebook content includes:

- Mathematical Foundations of Computer Graphics and Vision 2023**
- Exercise 1 - Robust Estimation and Optimization**
- Introduction text: "In this exercise you will apply what you learned about robust optimization, especially RANSAC and Iteratively Reweighted Least Squares (IRLS). You will implement RANSAC for circle fitting in the presence of outliers, IRLS for line fitting with L_1 norm, and LP for line fitting with L_1 and L_∞ norms. We have implemented a skeleton notebook and you have to fill out blocks marked with 'TODO'. Please refrain from modifying the notebook outside of the 'START' and 'END' blocks for the coding problems apart from new cells you create. Use numpy vectorization instead of python loops where explicitly noted. We may deduct points if your code runs unreasonably slow due (see the 'test case' cell)."
- Part 1: RANSAC for circle fitting**
- 1.1. Data Generation and Model Fitting**

```
In [1]: In [1]: import matplotlib inline
import math
from typing import Tuple, List, Optional
import itertools
from collections import namedtuple

import numpy as np
from scipy.optimize import linprog
import matplotlib.pyplot as plt

np.random.seed(123)
```

```
In [2]: In [2]: # Default parameters
N = 100 # number of total points
D = 10 # half-domain size
tau = 0.1 # inlier distance threshold
```

```
In [3]: In [3]: class Polynomial:
    def __init__(self, coefficients):
        self._coefficients = coefficients

    def __repr__(self):
        return f"Polynomial({self._coefficients.tolist()})"

    @property
    def degree(self):
        return len(self._coefficients) - 1

    def evaluate(self, x):
        """
        x - (N,1) array with the 'x' coordinates
        Returns - (N,1) array with their 'y' coordinates
        """
        y = np.ones_like(x)

        # TODO: evaluate the polynomial at points x
        # Use numpy vectorization!
        # START

        # END

        return y

    def is_inlier(self, points: np.ndarray, threshold: float = tau):
        """
        points - Nx2 array with the data points
```

Questions?